CST8509 Lab3 Gazebo

Overview

Gazebo is a collection of opensource libraries that facilitates robotics development and more. We will use the Gazebo simulator as a tool for applying Reinforcement Learning techniques to our real-world Create 3 robot application.

In this lab you will learn how to deploy your own simulation environment specifically for your Create 3 robot, in preparation for RL training do be done later in Assignment 2.

When you have completed this lab, you will know how to

- Install the iRobot Create 3 simulator
- Install the AWS small house world
- Run the iRobot Create 3 simulation, view it in the Gazebo UI and monitor it with RViz
- Add a virtual "loaner laptop" camera to the Create 3 simulation
- Configure the virtual camera to publish its virtual images to a ROS 2 topic
- Use command line to control the simulated Create 3 to navigate the AWS small house and monitor its virtual camera

Instructions

The primary development platform for this lab is your Ubuntu 22.04 loaner laptop. You can possibly also complete this work on other platforms, such as your Ubuntu 22.04 virtual machine, but if you encounter issues you'll need to be prepared to fall back to your loaner laptop.

Versions

Gazebo versions can be confusing, because there are two choices:

- Classic Gazebo (version 11), or Gazebo11 or Gazebo-11 or Classic Gazebo
- Ignition Gazebo (version Fortress, Harmonic, etc). Note that Ignition Gazebo has been renamed to just Gazebo! This Gazebo choice can be called Gazebo, Ignition Gazebo, Gazebo Sim.

We will use Classic Gazebo version 11.

The iRobot Create 3 simulator repository branch we will use provides instructions for two versions of Gazebo:

- for Classic Gazebo Version 11, which works with the AWS small house world
- for Ignition Gazebo Fortress (we will ignore this one for now)

We are using Classic Gazebo version 11, so we will use the instructions for the Classic version.

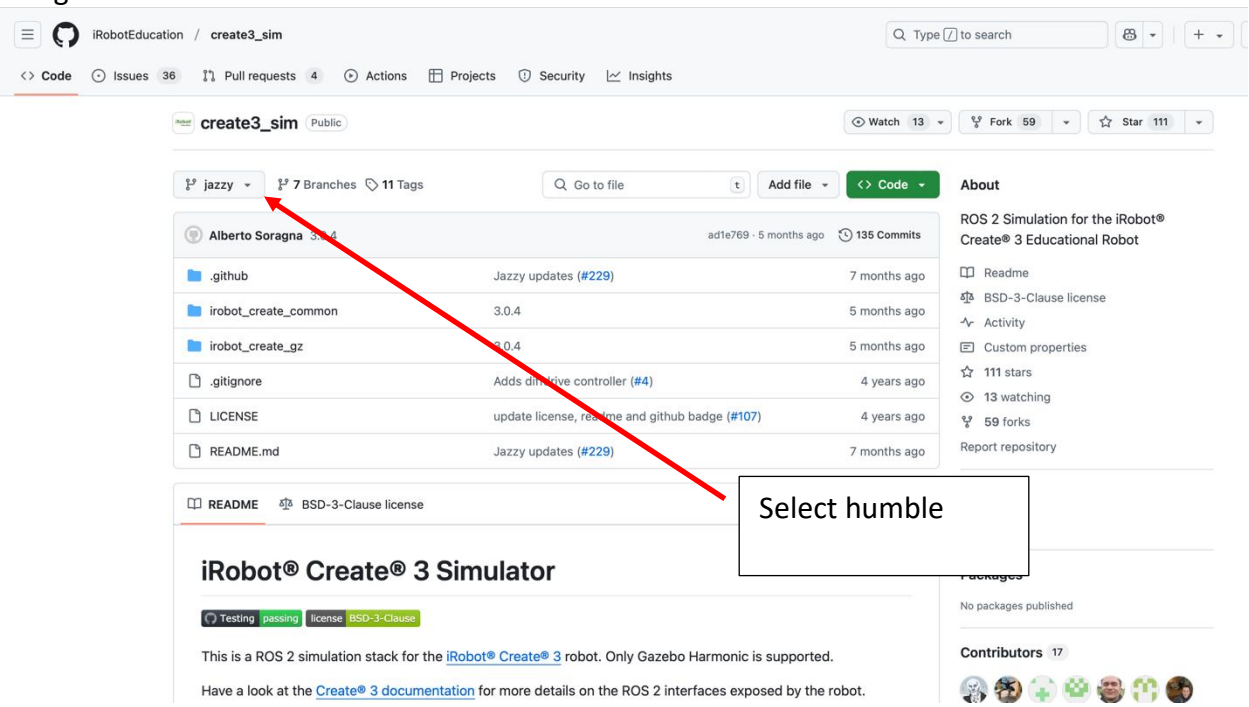
Install the Ubuntu Desktop GUI on your loaner laptop

To install the Ubuntu desktop version, with GUI, on your loaner laptop, issue the following command:

sudo apt install ubuntu-desktop

Deploy Gazebo11 Create3 simulator

The repository for the iRobot Create 3 simulator is here (we will need to use the humble branch): https://github.com/iRobotEducation/create3_sim To view the humble branch, select it using the branch selector:



Read this paragraph and its bullets, and then carefully follow the Classic Gazebo instructions given by the README of that repository. Note that:

- When you clone the repository, you will need to check out the humble branch:
  cd ~/create3_ws/src/create3_sim
  git checkout humble
- You need to ensure the prerequisites are installed, and pay attention to the links given in the README. One of the prerequisites is ROS 2 Humble, and now we will need the desktop version of ROS2 Humble. If you have already set up ROS 2 Humble (CST8504), then you can do just this:
  - sudo apt install ros-humble-desktop
- On the Humble branch, there are two "streams" in the instructions of the README, and you want to follow the **Classic Gazebo** stream (not Ignition Fortress).

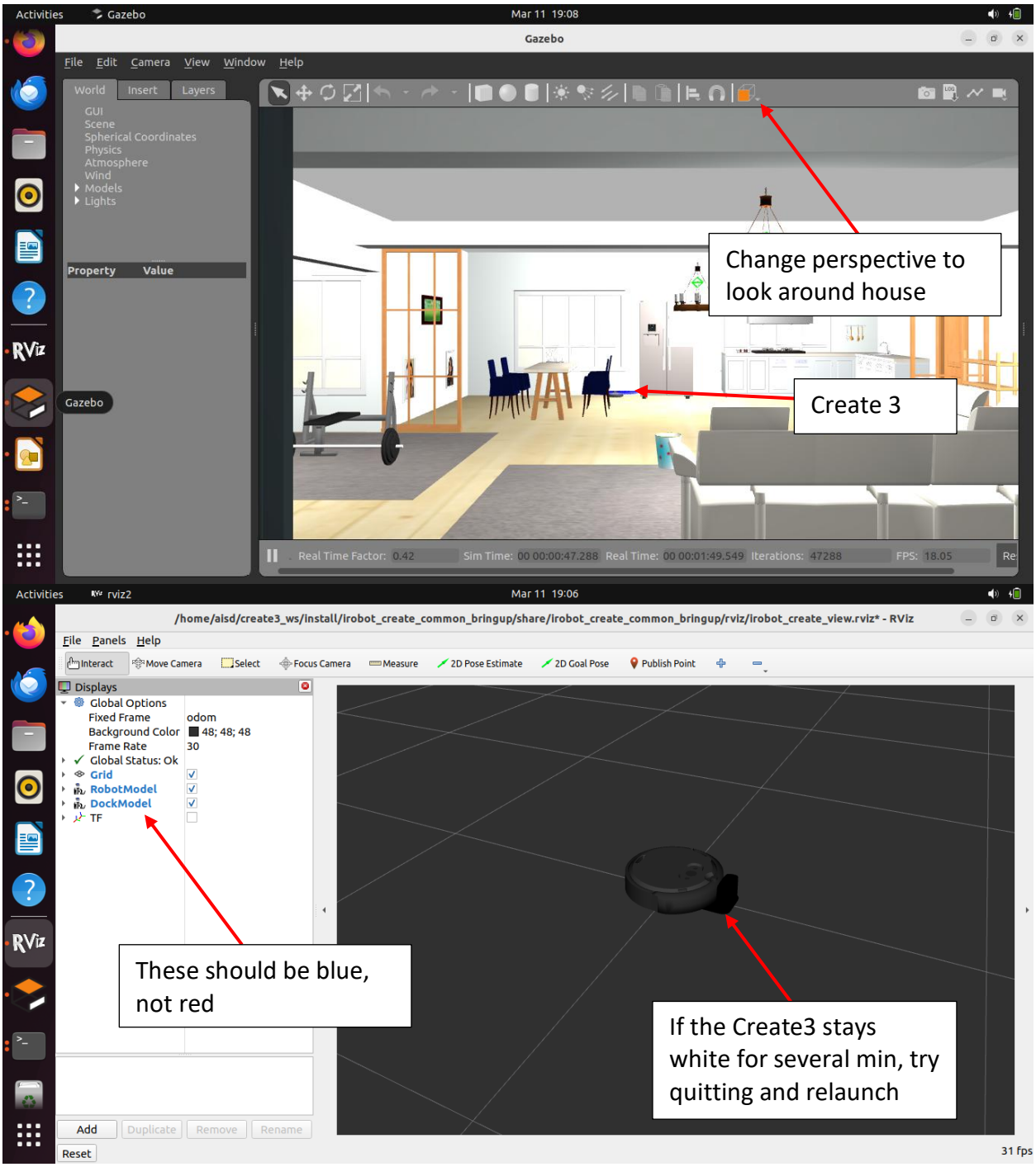
- To install Classic Gazebo 11 on Ubuntu 22.04, use the following command
    - curl -sSL http://get.gazebosim.org | sh

  or, alternatively
    - sudo apt install gazebo
- Under the Build section of the README, note that you are building a ROS 2 workspace using basically the same process you used to build your ROS2 workspaces in CST8504. Don't forget the following steps:
    - Clone the create3_sim GitHub repository into your workspace's **src** directory and don't forget to checkout the humble branch:
      cd ~/create3_ws/src/create3_sim
      git checkout humble
    - The **export IGNITION_VERSION=fortress** part is fine, even for Classic Gazebo
    - Run the given rosdep command. (sometimes easy to forget this command)
    - Run the given colcon build command. (sometimes easy to forget this command)

Run simulator with AWS house

Follow the instructions in the README for downloading the source code and building the AWS small house world.
After the AWS small house world is built, you can run the AWS small house launcher command given in the instructions. Be patient when the programs are launching. Sometimes the GUI will be "not responding", but waiting a few minutes should be all that's needed. If you've waited several minutes and Rviz shows the Create3 as white, with errors on the left pane, you can try ^C in the terminal where you launched, and then re-run the launcher.

You should then see the Gazebo GUI showing the AWS house with the Create3 in the distance, and the RViz GUI showing the Create3 in its dock:



At this point, the Create 3 simulation will function very much like your actual Create 3 with your loaner laptop. There is no need for the network cable in this case because the simulated Create 3 is already on the same network as your loaner laptop – the loaner laptop and simulated Create 3 both see the same ROS 2 topics. You can issue the docking and undocking commands

on the loaner laptop command line, and the Create 3 simulation will dock/undock in its virtual world.
Optional: The simulated Create 3 will also move with your aisd_vision code from CST8504, the same way the turtlesim simulator did. Your aisd_vision code will publish images from your loaner laptop camera, and publish the hand commands, which in turn result in movement (Twist) messages, which in turn will cause the Create 3 simulation to move like a real Create 3.

Add Virtual Camera to Create 3

The next step to prepare for Reinforcement Learning training is to add a virtual camera to the simulated Create 3, so when the Create 3 moves, the camera moves with it, and the camera sees what the loaner laptop camera would see if it were actually in the AWS small house.

The parts of the simulated Create 3 robot are specified in the Unified Robotic Description Format (URDF), which is an XML file format used in ROS to describe all elements of a robot. Additionally, there is a new format called the Simulation Description Format (SDF) which was created for use in Gazebo to solve the shortcomings of URDF. For our purposes in this lab, we can rest assured that the URDF files will automatically be converted to SDF. Lastly we will also encounter Xacro, which is an XML macro language (both URDF and SDF are XML languages). We will not take the time to focus on the details of URDF, SDF, or Xacro, but for those who want to dig deeper, there is a tutorial available here:
https://classic.gazebosim.org/tutorials?tut=ros_urdf#

The URDF files for the Create 3 simulation are here:
~/create3_ws/src/create3_sim/irobot_create_common/irobot_create_description/urdf/

The following file is the starting point for the Create 3 robot:
~/create3_ws/src/create3_sim/irobot_create_common/irobot_create_description/urdf/create3.urdf.xacro

This file has a <robot> element that includes the parts of the robot:

```
<?xml version="1.0" ?>
<robot name="create3" xmlns:xacro="http://ros.org/wiki/xacro">
  <xacro:include filename="$(find irobot_create_description)/urdf/bumper.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/button.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/caster.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/common_properties.urdf.xacro"/>
  <xacro:include filename="$(find irobot_create_description)/urdf/sensors/cliff_sensor.urdf.xacro"/>
  <xacro:include filename="$(find irobot_create_description)/urdf/sensors/imu.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/sensors/ir_intensity.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/sensors/ir_opcode_receivers.urdf.xacro" />
  <xacro:include filename="$(find irobot_create_description)/urdf/sensors/optical_mouse.urdf.xacro"/>
  <xacro:include filename="$(find irobot_create_description)/urdf/wheel_with_wheeldrop.urdf.xacro" />


  <!-- Gazebo version -->
    <xacro:arg name="gazebo"                default="classic" />
```

…continued..

We can see from the filename highlighted in red how the wheel_with_wheeldrop component is added to the robot. We will add another line the same as the one for wheel_with_wheeldrop,

right under it, except the filename will be a camera URDF file we will create, called **camera.urdf.xacro**

Now we need to create the **camera.urdf.xacro** file which will go in that same directory of URDF files. The following instructions are a summary of the tutorial here:
https://articulatedrobotics.xyz/mobile-robot-9-camera
We start with a <robot> element, and add a joint (for the camera to attach to the Create 3) and a link (the camera object itself is a "link"). The joint (camera_joint) is between the Create 3 (base_link) and the camera itself (camera_link). The camera is made of a red material, so we also define that material with red color.

```
<?xml version="1.0"?>
<robot xmlns:xacro="http://www.ros.org/wiki/xacro" >

   <joint name="camera_joint" type="fixed">
      <parent link="base_link"/>
      <child link="camera_link"/>
      <!-- the camera will sit 0.2 m above the create3 base_link -->
      <origin xyz="0 0 0.2" rpy="0 0 0"/>
   </joint>

   <link name="camera_link">
      <visual>
         <geometry>
            <!-- the dimensions of the camera box -->
            <box size="0.010 0.03 0.03"/>
         </geometry>
         <material name="red"/>
      </visual>
   </link>
   <material name="red">
      <color rgba="1 0 0 1"/>
   </material>
```

Now, the standard ROS robot orientation is (x-forward, y-left, z-up) and we will want the standard optical orientation (x-right, y-down, z-forward), so we add another link with a joint. The purpose of this new link (camera_link_optical) is to point in the z-forward direction:

```
   <joint name="camera_optical_joint" type="fixed">
      <parent link="camera_link"/>
      <child link="camera_link_optical"/>
      <origin xyz="0 0 0" rpy="${-pi/2} 0 ${-pi/2}"/>
   </joint>
```

```
  <link name="camera_link_optical"></link>
```

At this point our camera is added to the robot, and we now need to make the camera function in the Gazebo simulated world. We do this with a <gazebo> element that includes a camera-type sensor:

```
  <gazebo reference="camera_link">
        <material name="red"/>
    <sensor name="camera" type="camera">
        <pose> 0 0 0 0 0 0 </pose>
        <visualize>true</visualize>
        <update_rate>10</update_rate>
        <camera name="head">
          <horizontal_fov>1.089</horizontal_fov>
          <image>
            <format>R8G8B8</format>
            <width>640</width>
            <height>480</height>
          </image>
          <clip>
            <near>0.05</near>
            <far>8.0</far>
          </clip>
        </camera>
```

The code needs finishing, but in principle we now have a functioning camera in Gazebo, and we want the images from the camera to be published to ROS 2 over a topic. We arrange for that with the **libgazebo_ros_camera** plugin (and then we close the <sensor>, <gazebo>, and <robot> elements):

```
 <plugin name="camera_controller" filename="libgazebo_ros_camera.so">
   <ros>
     <namespace>custom_ns</namespace>
     <remapping>image_raw:=custom_img</remapping>
     <remapping>camera_info:=custom_info</remapping>
   </ros>
   <camera_name>camera1</camera_name>
   <frame_name>camera_link_optical</frame_name>
   <hack_baseline>0.7</hack_baseline>
 </plugin>
 <!-- fix indentation -->
        </sensor>
```

```
  </gazebo>
</robot>
```

With the camera.urdf.xacro file completed (and included from create3.urdf.xacro), build the workspace:

```
colcon build --symlink-install --packages-select irobot_create_description
```

Verify the Camera is working

Shut down the previous run (without the camera) by typing ^C (ctrl-C) in the terminal window where you ran the launch command.

IMPORTANT: In order for the Gazebo camera plugin to work, we need to do the following

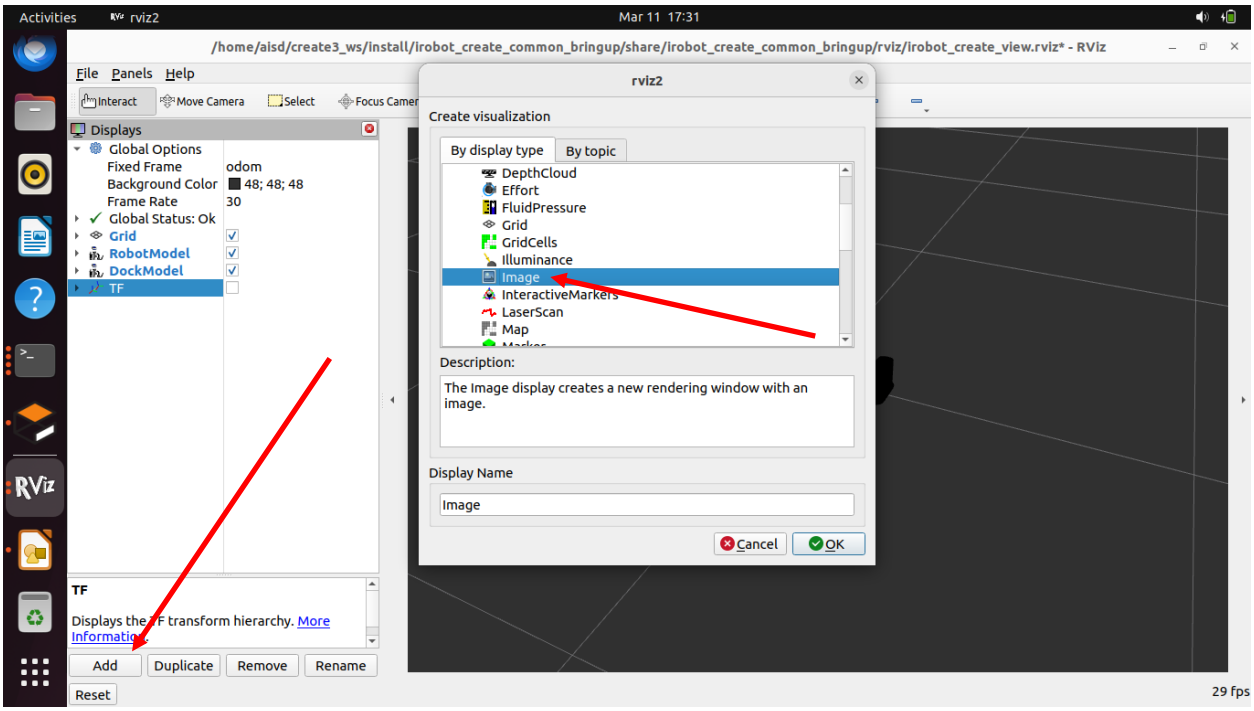
```
source /usr/share/gazebo-11/setup.sh
```

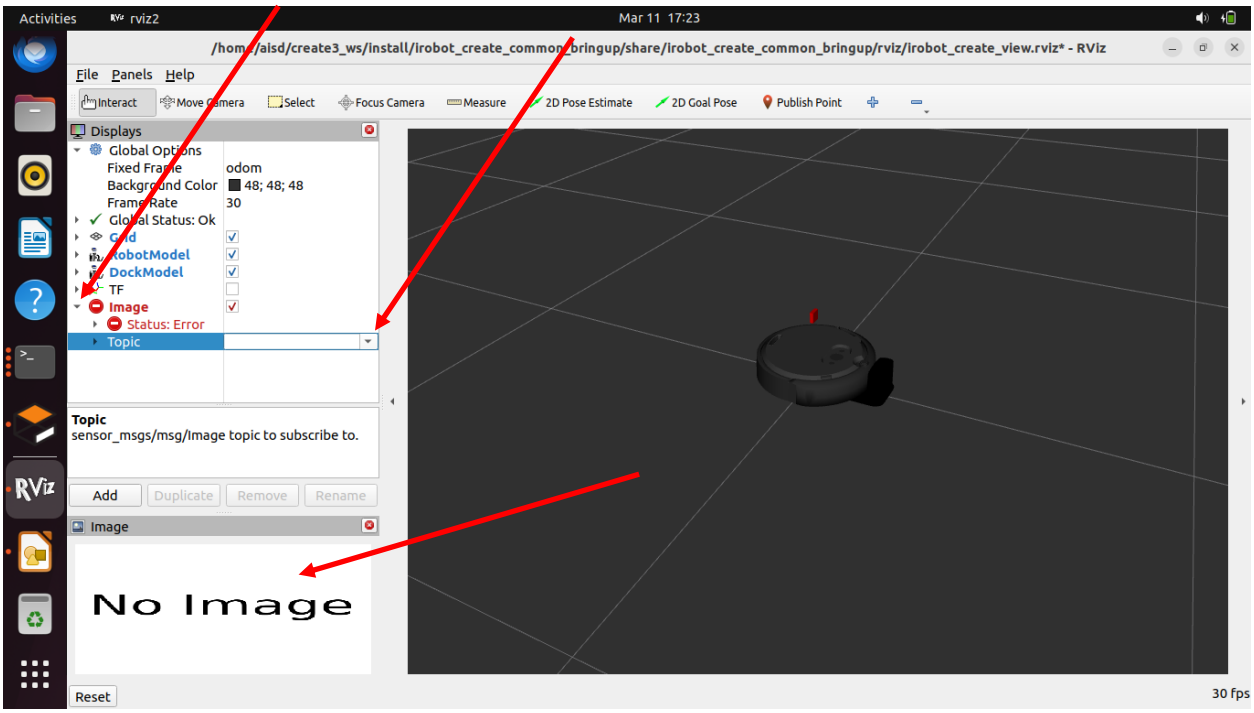
Without this step, the ROS 2 topics for the camera will not appear in **ros2 topic list**. You could add this line to the bottom of your ~/.bashrc file to run the command every time you log in:

echo source /usr/share/gazebo-11/setup.sh >> ~/.bashrc

Now, rerun the command to launch the simulator with the AWS small house. Check your **ros2 topic list** to see that your camera topics are now present.

Using the Rviz GUI, add an **Image** to display the published images coming from the topic we set up for the Gazebo camera: **/custom_ns/camera1/custom_img :**





After adding the image, you can now see what the virtual camera sees in the virtual world in the Image pane in the lower left corner of the Rviz GUI. Using the command line https://iroboteducation.github.io/create3_docs/examples/actuators-cli/ you can undock the Create 3 and navigate it around the AWS small house, viewing the camera in Rviz.

You are now ready for the future assignment where we will use our setup to do RL training on our simulated Create 3.

Submission

Submit your **camera.urdf.xacro** file to Brightspace.

Demonstration

- Show your lab instructor your running simulation with the camera added to the Create3.
- Run commands to undock the create3 and drive it around the AWS small house.
- Show the camera image being simulated and projected in the Gazebo GUI
- Show the camera image being monitored in Rviz